COMPLETE DSA PATTERNS & ALGORITHMS

Beginner to Advanced • A structured learning reference for coding interviews and algorithmic problem solving

What this guide covers

- Foundations: complexity, mathematics, sorting, searching and recursion.
- Core interview patterns: hashing, two pointers, sliding window, prefix sums, binary search, stacks, queues and heaps.
- Problem-solving families: greedy, backtracking, divide and conquer and dynamic programming.
- Trees, graphs, strings, bit manipulation, number theory and advanced data structures.
- A practical 24-week learning sequence, recognition guide and mastery checklist.

How to use this PDF

Study one pattern at a time. For each pattern, learn the invariant or recurrence, implement the basic algorithm, solve several problems, and then solve a new problem without looking at the solution. The topic list is comprehensive, but advanced topics are prioritized selectively for interview preparation.

Prepared as a study reference • Includes algorithm families, recognition clues, example problem types, complexity notes and a suggested study order.

Contents

Use this map to navigate the learning sequence. The sections progress from foundational concepts to specialized advanced topics.

Section	Topic
01	The DSA mental model and complexity
02	Programming foundations, mathematics and basic algorithms
03	Arrays, strings and hashing patterns
04	Linked lists, stacks, queues and deques
05	Binary search, heaps, intervals and greedy algorithms
06	Recursion, backtracking and combinatorial search
07	Dynamic programming patterns
08	Trees and binary search trees
09	Graph algorithms and connectivity
10	Strings, tries and pattern matching
11	Bit manipulation and number theory
12	Range-query structures and advanced data structures
13	Specialized algorithms
14	Pattern combinations and recognition guide
15	24-week learning sequence
16	Mastery checklist and problem-solving workflow

A note on scope: no single roadmap can enumerate every algorithmic technique. This guide covers the main families used in general software engineering interviews and adds advanced topics commonly found in competitive programming and specialized algorithm work.

01. The DSA mental model

DSA is not just a list of data structures. The skill is identifying the structure of a problem, choosing an appropriate technique, and explaining why the solution is correct.

Term	Meaning	Examples
Data structure	How information is organized and accessed.	Array, hash map, heap, tree, graph
Algorithm	A defined procedure for solving a problem.	Binary search, Dijkstra, merge sort
Pattern	A reusable approach for a family of problems.	Sliding window, two pointers, backtracking
Invariant	A property that remains true while an algorithm runs.	Sorted search boundary; monotonic stack order
State / recurrence	A compact description of a subproblem and how it depends on others.	Dynamic programming state and transition

Time complexity

Complexity	Typical example	Interpretation
$O(1)$	Array access by index	Constant work
$O(\log n)$	Binary search	Repeatedly reduce search space
$O(n)$	Single traversal	Work proportional to input size
$O(n \log n)$	Merge sort	Divide-and-conquer sorting
$O(n^2)$	Compare every pair	Quadratic growth
$O(2^n)$	Enumerate all subsets	Exponential growth
$O(n!)$	Enumerate all permutations	Factorial growth

Space complexity and analysis

- Distinguish auxiliary space from total memory usage.
- Account for recursion stack depth and temporary data structures.
- Know in-place algorithms, amortized analysis and time-space trade-offs.
- Consider worst-case, average-case and expected complexity where relevant.
- Use input constraints to estimate whether an algorithm is likely to pass.

02. Foundations, mathematics and basic algorithms

Mathematical foundations

- Divisibility, factors, multiples, primes and prime factorization.
- GCD and LCM; Euclidean algorithm and extended Euclidean algorithm.
- Sieve of Eratosthenes and smallest-prime-factor sieve.
- Modular arithmetic, modular exponentiation and modular inverse under valid assumptions.
- Fast exponentiation, Fibonacci fast doubling, logarithms and growth rates.
- Combinatorics: factorials, binomial coefficients, Pascal's triangle and inclusion–exclusion.
- Basic probability, expected value, pigeonhole principle and Catalan numbers.
- Bitwise AND/OR/XOR, shifts, set/clear/toggle/check bits.

Basic searching and sorting

Family	Algorithms / concepts	What to understand
Searching	Linear search, binary search, lower/upper bound	Preconditions, boundaries and complexity
Elementary sorting	Bubble, selection, insertion sort	Mechanics and $O(n^2)$ behavior
Efficient sorting	Merge sort, quicksort, heapsort	Partitioning, recursion, stability and complexity
Non-comparison sorting	Counting sort, radix sort	When value/domain assumptions make them useful
Array operations	Reverse, rotate, merge, remove duplicates	In-place changes, indexing and edge cases

Do not spend disproportionate time memorizing elementary sorting code. Understand the mechanics and trade-offs, then focus on patterns that use sorting as a building block.

03. Arrays, strings and hashing patterns

These are high-value interview topics. Many apparently different questions are variations of a small number of techniques.

3.1 Traversal and in-place manipulation

Level: Beginner

Recognize it when: The task requires scanning, reversing, rotating, partitioning or modifying an array with limited extra memory.

Core techniques: Single pass, reverse traversal, swaps, cyclic rotation, partitioning.

Example problems / algorithms: Move zeroes, reverse array, rotate array, remove duplicates from a sorted array, Dutch National Flag.

3.2 Hashing and frequency counting

Level: Beginner

Recognize it when: You need fast membership checks, duplicate detection, frequency counts or value-to-index lookup.

Core techniques: Hash map, hash set, frequency array, canonical keys.

Example problems / algorithms: Two Sum, valid anagram, group anagrams, duplicate detection, frequency queries.

3.3 Two pointers

Level: Beginner → Intermediate

Recognize it when: A pair or sequence can be processed from opposite ends or by maintaining two positions.

Core techniques: Left/right pointers, same-direction pointers, slow/fast pointers, sorted pair search.

Example problems / algorithms: Two Sum in sorted array, Three Sum, palindrome check, container with most water, in-place filtering.

Important note: Pointer movement must be justified by sorted order or another structural property.

3.4 Sliding window

Level: Intermediate

Recognize it when: The target is a contiguous subarray or substring and a maintained window condition can be updated as boundaries move.

Core techniques: Fixed-size window, variable-size window, frequency-maintained window.

Example problems / algorithms: Maximum sum of size k, longest unique substring, minimum window substring, longest qualifying subarray.

Important note: Variable-window shrinking is not generally valid with negative values unless the condition supports the movement.

3.5 Prefix sums, suffix sums and prefix XOR

Level: Intermediate

Recognize it when: You need repeated range aggregates or to reason about the portion before/after an index.

Core techniques: Cumulative sums, prefix frequency, suffix aggregation, prefix XOR.

Example problems / algorithms: Range-sum queries, equilibrium index, subarray sum equals k, product except self, XOR range queries.

3.6 Kadane's algorithm and running state

Level: Intermediate

Recognize it when: You need the best contiguous segment and can express the best result ending at the current position.

Core techniques: Extend the current segment or restart; track current maximum/minimum where needed.

Example problems / algorithms: Maximum subarray sum, maximum circular subarray sum, maximum product subarray.

3.7 Difference arrays

Level: Intermediate

Recognize it when: Many updates add a value across a range, and answers are needed after all updates.

Core techniques: Record changes at interval boundaries, then prefix-scan.

Example problems / algorithms: Range increment updates, booking counts, offline range modifications.

3.8 Sweep line and offline queries

Level: Intermediate → Advanced

Recognize it when: Events can be sorted and processed in order, or queries can be reordered to reduce repeated work.

Core techniques: Sort events, maintain active state, coordinate compression, offline processing.

Example problems / algorithms: Maximum overlapping intervals, count values under thresholds, range/event queries.

3.9 Cyclic placement

Level: Intermediate

Recognize it when: Values are constrained to a known range and can be placed at corresponding indices.

Core techniques: Index-value correspondence and in-place swapping.

Example problems / algorithms: Find missing/duplicate values under constrained input assumptions.

3.10 Meet in the middle

Level: Advanced

Recognize it when: Brute force over all subsets is too expensive, but splitting the input into two halves is feasible.

Core techniques: Enumerate each half, store/sort partial results, combine complementary values.

Example problems / algorithms: Subset sum and optimization problems with moderate n.

Pattern combination example: subarray sum equals k

Maintain a running prefix sum s. Any earlier prefix sum equal to s − k identifies a subarray ending at the current index with sum k. Store prefix-sum frequencies in a hash map. This combines prefix sums and hashing and works with negative values; expected time is O(n).

04. Linked lists, stacks, queues and deques

Linked-list patterns

Fast and slow pointers

Level: Beginner → Intermediate

Recognize it when: You need a midpoint, cycle detection or a relationship between positions.

Core techniques: Move pointers at different speeds; use Floyd's tortoise-and-hare algorithm.

Example problems / algorithms: Middle node, cycle detection, cycle entry, palindrome list.

Reversal and pointer rewiring

Level: Beginner → Intermediate

Recognize it when: Links can be changed to reverse or reorder nodes without copying values.

Core techniques: Maintain previous/current/next pointers; reverse recursively or iteratively.

Example problems / algorithms: Reverse list, reverse sublist, reverse nodes in groups of k.

Dummy/sentinel node

Level: Beginner → Intermediate

Recognize it when: Head deletion or insertion creates awkward special cases.

Core techniques: Use a temporary node before the real head.

Example problems / algorithms: Remove nth node from end, merge sorted lists, delete matching nodes, partition list.

Merge and intersection

Level: Intermediate

Recognize it when: Two or more lists must be combined or share a node.

Core techniques: Two-pointer merge, pointer alignment, divide and conquer or heap.

Example problems / algorithms: Merge two lists, merge k lists, intersection of lists, reorder list.

Complex pointer copying

Level: Intermediate → Advanced

Recognize it when: Nodes include random or non-next references.

Core techniques: Hash-map association or interleaving clone method.

Example problems / algorithms: Copy list with random pointer.

Stack patterns

Pattern	Invariant / approach	Example problems
Basic stack	Last-in, first-out state	Valid parentheses, undo
Monotonic increasing stack	Maintain increasing keys/values as defined	Next smaller element, histogram
Monotonic decreasing stack	Maintain decreasing keys/values as defined	Next greater element, stock span
Expression evaluation	Process operators and precedence	Postfix evaluation, calculators
Boundary contribution	Resolve nearest smaller/greater boundaries	Sum of subarray minimums
Nested parsing	Track structure and matching delimiters	Nested strings, bracket matching

A monotonic stack can be O(n) even when its code contains a nested while loop: each element is pushed once and popped at most once.

Queue and deque patterns

- Queue implementation, circular queue, queue using stacks and stack using queues.
- BFS traversal and multi-source BFS.
- Monotonic deque for sliding-window maximum/minimum in O(n).
- 0–1 BFS using a deque for graphs with edge weights of zero or one.

05. Binary search, heaps, intervals and greedy algorithms

Binary search families

Pattern	Core idea	Examples
Classic binary search	Eliminate half a sorted search range	Find target, insertion position
Lower/upper bound	Find first index satisfying a boundary condition	First/last occurrence, count duplicates
Rotated / structured arrays	Use partial order or shape properties	Rotated search, minimum, bitonic array
Binary search on answer	Search a monotonic feasibility condition	Minimum shipping capacity, minimum speed, maximize minimum distance
Real-valued search	Narrow a continuous range to tolerance	Numerical roots, geometric approximation
Implicit/2D search	Exploit monotonic structure without flattening blindly	Sorted matrix, median of two sorted arrays

Correctness test: prove the predicate is monotonic. A minimum/maximum question alone does not imply binary search is valid.

Heap and priority-queue patterns

Pattern	Use	Example problems
Top-K / bounded heap	Keep only the best k candidates	K largest elements, kth largest
Two heaps	Maintain lower and upper halves	Running median
K-way merge	Repeatedly take the next smallest head	Merge k sorted lists
Repeated best-choice extraction	Choose current minimum/maximum efficiently	Huffman coding, combine ropes
Heap + greedy	Select best available item under changing constraints	Refueling stops, course scheduling
Heap + graph	Prioritize tentative distance or candidate edge	Dijkstra, Prim

Intervals and sweep line

- Merge intervals: sort by start and merge overlaps.
- Activity selection: sort by finishing time to maximize compatible activities.
- Meeting rooms: sort starts and track active end times with a min-heap.
- Sweep line: turn intervals into ordered start/end events and maintain active state.
- Difference events and coordinate compression: turn range operations into boundary changes or compact ranks.

Greedy algorithms

Greedy makes a locally preferred choice at each step. It is correct only when that choice can be justified globally.

- Earliest finishing time: activity selection and non-overlapping intervals.
- Reachability frontier: Jump Game and related reachability problems.
- Coverage: minimum arrows or intervals to cover points.
- Greedy + heap: refueling and deadline/selection problems.

- MST greedy: Kruskal and Prim.
- Correctness tools: exchange arguments, greedy-choice property, cut/safe-edge property.

06. Recursion, backtracking and combinatorial search

Recursion fundamentals

- Base case, recursive reduction and function contract.
- Linear recursion, multiple recursive calls and recursion trees.
- Divide and conquer: solve subproblems, then combine results.
- Tree recursion and postorder aggregation.
- Analyze number of calls, stack depth and total work.

Backtracking patterns

Pattern	Choice model	Example problems
Subsets / subsequences	Include or exclude each element	All subsets, subsequence generation
Permutations	Choose an unused element for next position	All permutations, constrained arrangements
Combinations	Choose values from a forward starting index	Combinations of size k, combination sum
Constraint satisfaction	Choose, validate, recurse, undo	N-Queens, Sudoku, graph coloring
Pruning / symmetry breaking	Stop impossible branches and skip equivalent choices	Duplicate-aware combinations, N-Queens
Branch and bound	Use optimistic bounds to stop branches that cannot beat best	TSP and assignment variants

Generic backtracking template

```
backtrack(state):
    if state is a complete solution:
        record or process the solution
        return

    for each valid choice:
        make the choice
        backtrack(updated state)
        undo the choice
```

Complexity can remain exponential: all subsets require $O(2^n)$ outputs and all permutations require $O(n!)$ outputs. Pruning can help but does not guarantee a better worst-case bound.

07. Dynamic programming (DP)

DP solves overlapping subproblems and reuses their answers. Do not start by memorizing solutions: define the state, derive the transition, establish base cases, choose evaluation order, and identify the answer.

The five-step DP framework

- State: what does dp[i], dp[i][j] or another state mean?
- Transition: how is the state computed from smaller states?
- Base cases: what can be answered directly?
- Evaluation order: which dependencies must be computed first?
- Answer extraction: which state or aggregation is the final result?

Memoization caches recursive results. Tabulation computes states iteratively in dependency order. Optimize space only after the recurrence and implementation are correct.

DP family	State / structure	Representative problems
1D DP	State depends on a few earlier positions	Climbing stairs, house robber, min-cost climbing
Grid / 2D DP	State depends on neighboring cells or two indices	Unique paths, minimum path sum, maximal square
Knapsack / subset	Choose items under capacity or target	0/1 and unbounded knapsack, subset sum, partition, coin change
Subsequence / sequence	Choose ordered elements from one/two inputs	LIS, LCS, edit distance, shortest common supersequence
String DP	Prefixes, substrings or matching states	Palindrome partitioning, wildcard and regex matching
Stock DP	Position plus holding/transaction/cooldown state	Single/multiple transactions, fee, cooldown, at most k
Interval DP	State represents interval [l, r]	Matrix-chain multiplication, burst balloons
Tree DP	Aggregate child states	Tree independent set, path sum, rerooting
DAG / graph DP	Process states in topological order	Longest path or count paths in a DAG
Bitmask DP	State includes selected/visited subset	TSP, assignment, subset ordering
Digit DP	Process digits with tightness and properties	Count integers meeting digit constraints
Advanced optimization	Exploit mathematical transition structure	Divide-and-conquer DP, convex hull trick, Knuth optimization, SOS DP

Example: maximum sum without adjacent elements

Define dp[i] as the maximum sum obtainable from the first i elements. Either skip the current element or take it and add its value to the best result two positions earlier:

$dp[i] = \max(dp[i-1], a[i] + dp[i-2])$

Choose base cases consistent with whether the empty selection is allowed. The key skill is deriving the state definition and transition, not memorizing this recurrence.

08. Trees and binary search trees

Tree traversal patterns

Pattern	Method	Example problems
DFS preorder	Node, left, right	Serialization variants, root-first processing
DFS inorder	Left, node, right	Sorted BST traversal
DFS postorder	Left, right, node	Height, subtree aggregation, delete tree
BFS / level order	Queue by levels	Zigzag traversal, right-side view, minimum depth
Subtree aggregation	Return child results and combine	Height, diameter, balance, max path sum
Path tracking	Maintain path state during recursion	Root-to-leaf path sums

Advanced tree patterns

- Lowest common ancestor (LCA), predecessor and successor.
- Build a tree from traversals when values and traversal assumptions permit unique reconstruction.
- Serialize and deserialize a tree.
- Euler tour / entry-exit times to map subtrees to intervals.
- Binary lifting for ancestor and LCA queries in O(log n) per query after preprocessing.
- Rerooting DP for answers from every possible root.
- Heavy-light decomposition for certain path queries and updates.
- Centroid decomposition for specialized distance/path problems.

Binary search tree (BST)

- Search, insert, delete, minimum and maximum.
- Validate using lower/upper bounds and a consistent duplicate policy.
- Kth smallest/largest, predecessor/successor and BST LCA.
- Recover swapped values and build a balanced BST from sorted data.
- Understand AVL trees and red-black trees conceptually.

09. Graph algorithms and patterns

Graph foundations

- Directed/undirected, weighted/unweighted graphs; paths, cycles, degrees and connectivity.
- Adjacency list versus adjacency matrix; sparse versus dense trade-offs.
- Disconnected graphs, isolated vertices, visited state and edge representation.
- DAGs, connected components and graph modeling from real problems.

Traversal and connectivity

Pattern	Core technique	Typical problems
DFS	Explore deeply and backtrack	Components, cycle detection
BFS	Process distance layers	Shortest path in unweighted graph
Multi-source BFS	Initialize queue with multiple sources	Rotting oranges, nearest source
Grid traversal	Treat cells as vertices	Islands, flood fill, surrounded regions
Cycle detection	Parent tracking, recursion states or indegree	Directed/undirected cycles
Bipartite checking	Two-color BFS/DFS	Two-color graph feasibility
Topological sort	Indegree queue or DFS completion order	Course scheduling, build dependencies
SCC	Low-link or reverse-graph traversal methods	Mutual reachability groups

Shortest paths

Algorithm	When to use	Typical complexity
BFS	Unweighted/equal-cost edges	O(V + E)
Dijkstra	Non-negative edge weights	O((V + E) log V) with binary heap and adjacency list
Bellman–Ford	Negative weights; reachable negative-cycle detection	O(VE)
Floyd–Warshall	All-pairs shortest paths on moderate-size graphs	$O(V^3)$ time, $O(V^2)$ space
DAG shortest paths	Directed acyclic graph; negative edges are fine	O(V + E) after topological order
0–1 BFS	Weights only 0 or 1	O(V + E)

Minimum spanning trees and connectivity

- Kruskal: sort edges and use DSU to avoid cycles.
- Prim: grow the tree by repeatedly taking the cheapest crossing edge.
- DSU / Union-Find: connectivity queries and component merging; use path compression and union by size/rank.
- Bridges and articulation points via discovery and low-link values.
- Strongly connected components: Kosaraju and Tarjan.
- Eulerian paths/circuits, bipartite matching, max-flow/min-cut, Dinic, Edmonds–Karp and min-cost max-flow.

For general software engineering interviews, prioritize BFS/DFS, cycle detection, topological sorting, Dijkstra, DSU and MST before advanced flow or dynamic graph techniques.

10. Strings, tries and pattern matching

Pattern / algorithm	Core idea	Example problems
Frequency signatures	Count characters or canonicalize strings	Anagrams, grouping
String sliding window	Maintain substring constraints	Longest unique substring, minimum window
KMP / prefix function	Reuse matched-prefix information	Pattern search in text
Z-algorithm	Longest prefix match at each position	Prefix matching and occurrence queries
Rabin–Karp / rolling hash	Hash candidate substrings efficiently	Pattern search, repeated substrings
Manacher's algorithm	Compute palindrome radii	Longest palindromic substring
Trie	Store shared prefixes as paths	Prefix search, dictionary lookup
Binary trie	Traverse bit representations greedily	Maximum XOR pair
Aho–Corasick	Automaton for multiple patterns	Multi-keyword matching
Suffix array + LCP	Sort suffixes and compare neighbors	Longest repeated substring
Suffix automaton/tree	Compact substring relationships	Distinct and repeated substrings

Use a trie when shared prefixes are important. For simple exact membership, a hash set is often simpler. Rolling hashes can collide, so verify candidates or use a sufficiently robust approach when correctness requires it.

11. Bit manipulation and number theory

Bit manipulation

- AND, OR, XOR, NOT and shifts; set, clear, toggle and check bits.
- x & (x − 1) clears the lowest set bit; use it for bit counting and power-of-two tests with proper edge handling.
- XOR cancellation: x XOR x = 0 and x XOR 0 = x.
- Bitmasks for subsets, visited sets and compact states.
- Population count, Brian Kernighan's algorithm, bitsets and precomputed tables.
- Submask enumeration, Gray codes, parity, SOS DP and subset transforms.
- Binary trie for maximum/minimum XOR queries.

Number theory and combinatorics

- GCD/LCM, Euclidean and extended Euclidean algorithms.
- Prime testing, Sieve of Eratosthenes, SPF sieve and factorization.
- Modular exponentiation; modular inverse only when mathematical conditions hold.
- Fast doubling for Fibonacci; matrix exponentiation for linear recurrences.
- Combinations, Pascal's triangle, inclusion–exclusion and Catalan numbers.
- Chinese remainder theorem and advanced number-theoretic functions for specialized tasks.

Basic bit operations, GCD, sieves and modular arithmetic are broadly useful. Advanced transforms and number-theoretic techniques are more common in competitive programming.

12. Range queries and advanced data structures

Structure / algorithm	Best fit	Important note
Prefix sums	Static range sums	Simple preprocessing; O(1) range-sum query
Sparse table	Static idempotent range operations	O(1) query for suitable operations after preprocessing
Fenwick tree (BIT)	Point updates and prefix aggregates	Compact and usually simpler than a segment tree
Segment tree	Flexible range aggregation and point updates	Often O(log n) query/update
Lazy propagation	Range updates plus range queries	Defer updates to covered segments
Persistent segment tree	Historical versions and range order statistics	Memory grows with created versions
Merge-sort tree	Range queries over sorted values	Useful for counting values in a value interval
Wavelet tree/matrix	Range frequency and order statistics	Supports kth-smallest/count queries
Mo's algorithm	Offline queries with reversible state updates	Query ordering reduces repeated work
Sqrt decomposition	Block-based range queries and updates	Simpler alternative for some constraints
Ordered balanced tree	Dynamic ordered membership	Predecessor/successor; rank support depends on implementation
Interval tree	Find overlapping intervals	Useful for interval-search workloads

Advanced structures

- AVL trees, red-black trees, treaps and skip lists.
- B-trees and B+ trees for storage/database indexing concepts.
- Bloom filters and counting Bloom filters for approximate membership.
- Persistent structures and implicit segment trees.
- Heavy-light decomposition, centroid decomposition and link-cut trees.
- Rope structures for editing long strings efficiently.

Selection guide

- No updates: prefix sums may be sufficient.
- Point updates + prefix/range sums: consider a Fenwick tree.
- Flexible aggregation or range updates: consider a segment tree.
- Static queries with a suitable idempotent operation: consider a sparse table.
- Offline queries with cheaply maintainable state: consider Mo's algorithm.

13. Specialized advanced algorithms

Computational geometry

- Orientation tests using cross products; segment intersection.
- Convex hull: monotonic chain and Graham scan.
- Closest pair of points and sweep-line geometry.
- Point-in-polygon, rotating calipers and spatial range searching.

Randomized algorithms and approximation

- Randomized quicksort/selection, reservoir sampling and randomized hashing.
- Monte Carlo versus Las Vegas algorithms; probabilistic analysis.
- Approximation ratios for problems where exact optimization is expensive.

Optimization, combinatorics and specialized graphs

- Bipartite matching: Hopcroft–Karp and assignment algorithms such as Hungarian.
- Maximum flow/minimum cut: Dinic, Edmonds–Karp and min-cost max-flow.
- Exact cover and Algorithm X; branch and bound.
- FFT/NTT and polynomial multiplication.
- Offline dynamic connectivity and DSU rollback.
- Advanced tree and graph decomposition: centroid decomposition, HLD and link-cut trees.

These are advanced extensions, not prerequisites for every entry-level interview. Prioritize them when your target role, contest goals or problem constraints justify the investment.

14. Pattern combinations and recognition guide

Problem shape	Patterns to consider	Why they fit
Longest substring with constraints	Sliding window + frequency map	Window state changes incrementally
Count subarrays with sum k	Prefix sum + hashing	Earlier prefix sum s − k identifies a match
Three Sum	Sorting + two pointers	Fix one value and search pairs efficiently
Largest rectangle in histogram	Monotonic stack	Find nearest smaller boundaries
Sliding-window maximum	Monotonic deque	Maintain only useful candidates
Merge k sorted lists	Linked lists + min-heap	Repeatedly select the smallest current head
Minimum feasible capacity	Binary search on answer + greedy check	Feasibility is monotonic
Course scheduling	Graph + topological sort	Dependencies define a directed graph
Minimum network connection cost	MST + DSU	Add safe low-cost edges without cycles
Word search across a dictionary	Backtracking + trie	Prune paths that are not dictionary prefixes
Longest increasing subsequence	DP or DP + binary-search optimization	Track best sequence states/tails
Kth smallest in a range	Persistent segment tree or wavelet structure	Support range order statistics
Maximum XOR pair	Bit manipulation + binary trie	Greedily choose opposite bits
Tree path queries	LCA, Euler tour or HLD	Transform ancestor/path relationships
Count integers by digit rules	Digit DP	State captures digit position and constraints
Many patterns in a text	Aho–Corasick	Process multiple patterns in one scan

How to recognize a pattern

- Contiguous subarray/substring: consider sliding window or prefix sums.
- Sorted data or monotonic feasibility: consider binary search or two pointers.
- Repeated membership/frequency lookups: consider hashing.
- Nearest greater/smaller element: consider a monotonic stack.
- Repeated minimum/maximum extraction: consider a heap.
- All possible valid arrangements: consider backtracking and pruning.
- Overlapping subproblems: define DP state and recurrence.
- Connectivity, dependencies or paths: model a graph.
- Repeated range queries: compare prefix sums, Fenwick and segment trees.

15. Suggested 24-week learning sequence

This is a practical sequence, not a guarantee that every advanced topic will be mastered in six months. Adjust the pace to your available study time and review needs.

Weeks	Focus	Checkpoint
1–4	Complexity, language fundamentals, mathematics, sorting, arrays, hashing, strings, two pointers	Solve basic array/string problems without viewing solutions
5–8	Sliding window, prefix sums, binary search, linked lists, stacks, queues, monotonic structures, heaps	Recognize and implement core linear patterns
9–12	Recursion, backtracking, trees, BSTs, greedy and LCA fundamentals	Implement traversals and explain recursive state
13–16	Graph modeling, BFS/DFS, cycles, topological sort, shortest paths, DSU, 1D/grid DP, knapsack	Select a traversal or DP state independently
17–20	Advanced DP, LIS/LCS, interval DP, tries, string matching, intervals, range-query fundamentals	Solve unfamiliar medium problems and explain trade-offs
21–24	Mixed-topic revision, weak patterns, selected advanced topics, timed assessments	Recognize patterns under time pressure and implement reliably

Suggested weekly practice rhythm

- Learn the pattern and implement a minimal example.
- Solve a small set of easy problems to build fluency.
- Solve medium problems that vary the same pattern.
- Re-solve missed problems after a delay without reading the solution.
- Keep a mistake log: misunderstanding, wrong invariant, edge case, complexity or implementation bug.

16. Mastery checklist and problem-solving workflow

Use this checklist as a progress record. Mark a topic complete only when you can explain the pattern, implement it, analyze complexity and solve a new problem independently.

Foundations

☐ Time and space complexity	☐ Basic mathematics and bit operations
☐ Sorting and searching	☐ Recursion fundamentals

Arrays and strings

☐ Array manipulation	☐ Hashing and frequency maps
☐ Two pointers	☐ Sliding window
☐ Prefix and suffix sums	☐ Kadane's algorithm
☐ Difference arrays	☐ Binary search on answer
☐ Divide and conquer	☐ Intervals and sweep line

Linear data structures

☐ Linked-list reversal and fast/slow pointers	☐ Stacks and queues
☐ Monotonic stack	☐ Monotonic deque
☐ Heap and priority queue	

Search and optimization

☐ Greedy algorithms and correctness	☐ Backtracking and pruning
☐ Subsets and permutations	☐ Meet in the middle

Trees and graphs

☐ Tree DFS and BFS	☐ BST operations
☐ LCA and binary lifting	☐ Graph traversal and components
☐ Topological sorting	☐ Shortest paths
☐ MST and DSU	☐ Bridges and SCCs
☐ Flow and matching	

Dynamic programming

☐ 1D and grid DP	☐ Knapsack and subset DP
☐ LIS and LCS	☐ String and interval DP
☐ Tree and DAG DP	☐ Bitmask and digit DP
☐ Advanced DP optimization	

Strings and advanced structures

☐ Trie and XOR trie	☐ KMP and Z-algorithm
☐ Rolling hash and Rabin–Karp	☐ Manacher's algorithm
☐ Fenwick tree and segment tree	☐ Sparse table and Mo's algorithm
☐ Persistent structures and advanced trees	☐ Computational geometry and randomized algorithms

Seven-step workflow for every problem

- Understand input, output, constraints and edge cases.
- Develop a correct brute-force baseline.
- Identify the structure: contiguous range, sorted order, repeated state, connectivity or constrained choices.
- Choose a candidate pattern and explain why it applies.
- State the invariant or recurrence explicitly.
- Prove correctness and analyze time/space complexity.
- Implement, test boundaries, record mistakes and revisit the problem later.

Final principle
The goal is not to finish the longest topic list. The goal is to look at an unfamiliar problem, identify a plausible approach, justify it, and implement it correctly without relying on a memorized solution.